안녕하십니까, 사용자의 불편을 화면과 서버에서 풀어온 지원자 김도현입니다.

서비스 운영에서 발견한 문제를 기능과 처리 구조의 개선으로 이어왔습니다.

현대오토에버에서 현업이 편리하게 사용하고 신뢰할 수 있는 플랫폼을 만들고 싶습니다.



Communication

소통 능력을 인정받아 팀을 끌어가는 자리에 나아갔습니다.

- 교내 앱 개발 동아리 FE 챕터 리드 및 운영진
- 교내 캡스톤 디자인 팀장
- kt cloud TECH UP 최종 프로젝트 개발 팀장 (기획, 디자인, 개발, 보안, 인프라, 총 15명 협업)

다양한 사람들과 만나 교류하며 성장하고 있습니다.

- 입학 후 교내 중앙 밴드 동아리 지속적 활동 공연 링크
- 교내 '드론활용 전문가 양성 교육' 참여, 항공 촬영 전문가 2급(민간인) 취득 촬영 영상 링크
- 동대문구 3개 대학 교류전 밴드 부문 우승
- 지구촌교회 분당채플 대학/청소년 지구 목자 섬김
- SSAFY 16기 서울캠퍼스 재학 중

Study

새로운 기술을 빠르게 익히고, 실제 개발에 적용하며 학습민첩성을 키워왔습니다.

FE

- React, TypeScript 관련 학습 및 앱/웹 개발
- '시대생' 앱 서비스 누적 다운로드 10,000건 기록
- '시대팅 시즌 6' DAU 1,000명, 결제 800건 달성

BE

- kt cloud TECH UP 백엔드 과정 1기 교육
- Java, Spring 기반 개발, Redis·Kafka 학습 및 활용

AI

- AI 에이전트·하네스 기반 개발 방식 학습
- ‘NaIQ’ 에이전트 역할 분리·작업 지침·자동 검증 적용

김도현 – 프로필

학력

- 서울시립대학교 컴퓨터과학부(19.03 – 26.08)
- GPA 3.54/4.5 (전공 학점 3.64)

수상내역

- 서울시립대학교 캡스톤디자인 우수작품상 수상 (25.01)

자격 및 어학

- 정보처리기사
- SQLD
- AWS SAA
- ADsP
- TOEIC Speaking IH(150)

대외 활동

1. 교내 앱 개발 중앙동아리 UOSLIFE (24.03 – 25.08)
   - 25-1학기 FE 챕터 리드
   - '시대생' 앱 운영
2. 반찬가게 픽업 플랫폼 맘찬픽 (24.08 – 25.09)
   - 팀장 및 FE 개발 담당
   - 서울시립대학교 캡스톤 디자인 우수작품상 수상
3. 42Seoul 9기 본과정 (23.03 – 23.11)
4. 교내 밴드 중앙동아리 THRASH AL (19.03 – 현재)

추가 교육

- 삼성청년SW·AI아카데미(SSAFY) (26.07 – 현재)
- kt cloud TECH UP 백엔드 과정 (25.09 – 26.04)

프로젝트 목차

아래 경험을 통해 오토에버에서 현업이 편리하게 쓰고 신뢰하는 플랫폼을 만들겠습니다.

프로젝트 소개 및 핵심 기여

---

1 시대생 – 서비스 운영

- 서울시립대학교 학사 정보 제공 앱 '시대생'
- 교내 앱 개발 동아리 'UOSLIFE' **FE 개발** 챕터
- TypeScript · React · React Native · Jotai

- '시대팅 시즌6' 운영 결과 결제 800건 달성
- 사용자 지표를 관측하며 꾸준한 서비스 개선
- **누적 다운로드 10,000건 이상 앱 운영**

04–06 ↗

---

2 Truve – BE

- 뮤지컬 팬덤을 위한 **MSA 기반** 예매 플랫폼
- 개발 팀장 및 **BE 개발** 담당 (kt cloud 교육과정 최종프로젝트)
- Java · Spring Boot · Redis · MySQL · Kafka

- Redis ZSET 기반 대기열 설계
- Outbox 패턴 활용 이벤트 발행·장애 복구
- JVM 웜업으로 초기 요청 지연 개선

07–09 ↗

---

3 NalQ – AI 활용

- 학습자료를 퀴즈와 복습으로 연결하는 플랫폼
- **AI 하네스 활용 개인 프로젝트**
- Java · Spring Boot · React · AWS

- AI 작업 맥락·역할·검증 기준 설계
- LLM 문제 생성 비동기 처리·상태 관리
- 웹 MVP 배포 및 운영 중

10–11 ↗

---

1. 시대생 UOSLIFE (24.03 – 25.08) UOSLIFE 팀페이지 Google play store App store

서울시립대학교 교내 학사 정보 제공 앱 | TypeScript · React · React Native · TanStack Query · Jotai
학교 생활의 불편을 줄이기 위해 교내 앱 개발 동아리 'UOSLIFE'가 개발하고 운영하는 서비스입니다.

개발이라는 기술을 통해 학우들의 불편을 개선하고자
학교 생활에 필요한 기능을 만들며 앱 운영에 참여했습니다.

2024년 연말에 시작된 서비스 리빌딩 기획에도 함께했습니다.
현재 리빌딩 이후 누적 다운로드 1만 건 이상을 기록하며,
학교 생활을 돕는 슈퍼앱으로 자리매김 하였습니다.

누적 다운로드 1만 건을 달성하는 여정에 FE 챕터로 아래와 같이 참여했습니다.

1 시간표 (24.12 – 25.08)

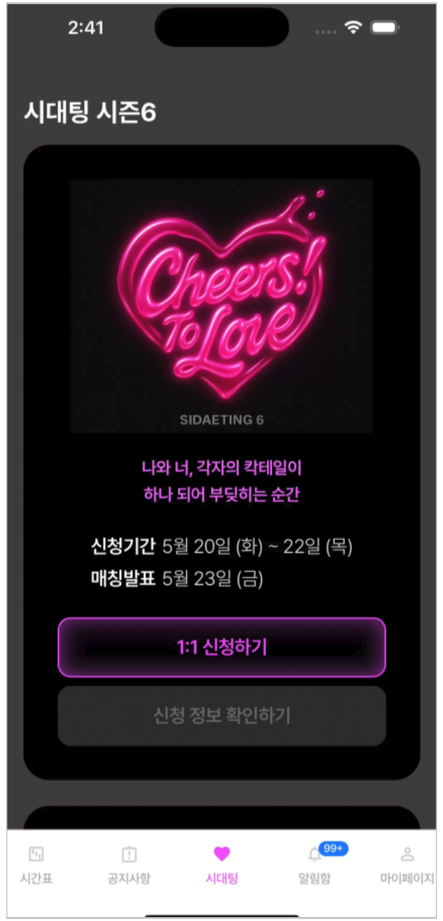
- UI와 비즈니스 로직을 커스텀 훅으로 분리
- 친구 관리·카카오톡 공유
- 플랫폼별 딥링크 전달 방식 개선

2 시대팅 시즌6 (25.03 – 25.06)

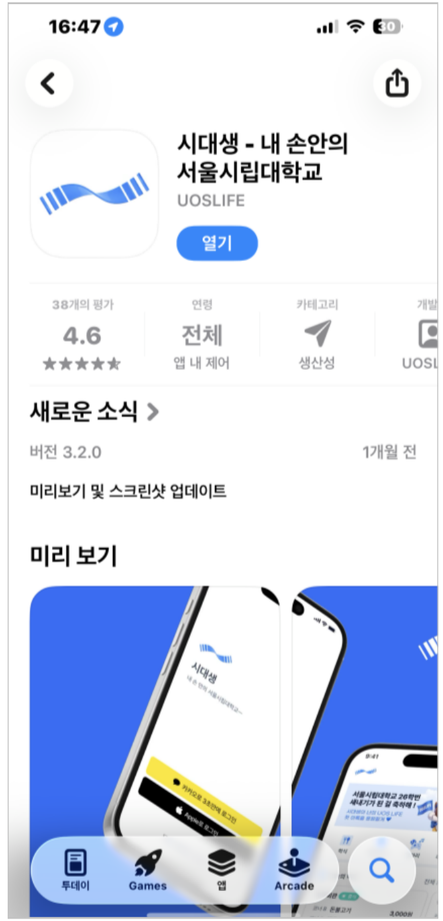
- 다단계 신청·퍼널 입력 상태 관리
- 앱 내 PortOne 결제 모듈 연동



▲시간표 화면



▲ 시대팅 입장 화면



▲ 앱스토어 배포

시대생 기여 사항 ① – 시간표 기능 및 공유 흐름 개선

UOSLIFE · '시대생' 앱 내 서비스 '시간표' / 2024.12 – 2025.08

WHY 개강 2주차 이후, 시간표 기능의 DAU가 급격하게 감소했습니다.

HOW

① 수강 과목 수에 따른 리텐션의 차이를 확인했습니다.
수강 과목이 적은 사용자군의 재방문율이 낮았습니다.

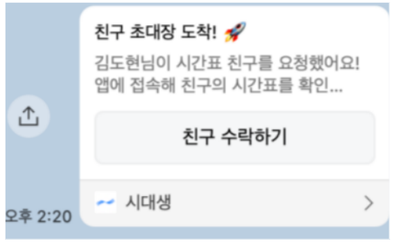
② 시간표에 익숙해져 서비스를 사용할 이유가 부족하다고 가정했습니다.

③ 친구 시간표 조회·공유를 재방문의 동기로 제안했습니다.
시간표를 소통의 장으로 확장시키고자 했습니다.

Solution 사용자 지표를 기준으로 아래과 같이 서비스를 개선했습니다.

친구 시간표 및 카카오톡 공유 구현

- 시간표 친구 관리·좋아요·공유 기능 구현
- 초대 링크에 친구 정보를 담아 딥링크로 연결



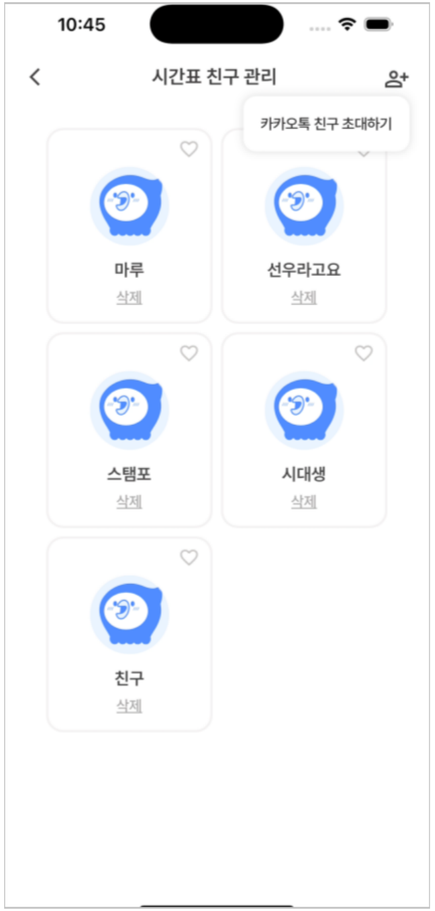
▲ 카카오톡 공유 메시지

OS별 메시지 처리 차이로 발생한 딥링크 오류 해결

- 안드로이드는 초대 링크 접속 시 메인 화면으로 접속되는 문제 발생
- 앱·웹 간 통신 문제로 좁히고, 안드로이드/iOS 간 메시지 처리 방식 파악
- OS별 메시지 수신 객체 분리, 웹뷰 로드 직후 브릿지 초기화 이벤트 전송
- 시대팅·공지사항 등 타 서비스에서도 활용할 수 있는 딥링크 기반 마련



▲ 시간표 메인 화면



▲ 친구 시간표 조회 화면

프로젝트 성과 | 친구 시간표 조회·공유 기능 배포 이후 서비스 WAU 30% 증가

목차로 이동

시대생 기여 사항 ② – 시대팅 시즌6 저속 네트워크에서 결제 복귀 흐름 개선

UOSLIFE · '시대생' 앱 내 서비스 '시대팅 시즌6' / 2025.03 – 2025.06 [인스타그램 마케팅 게시글]

WHY

'시대팅 시즌6' 는 교내 축제 기간에 서비스가 예정되어 있었습니다.

교내 네트워크는 축제 기간마다 사용량 증가로 속도가 저하되었습니다.

HOW

① iOS 시뮬레이터로 1KB/s 환경까지 점검했습니다.

대역폭을 낮추자 결제 이후 앱 복귀 시 화면 전환 오류가 발생했습니다.

② 딥링크 이벤트를 큐에 저장하고, 웹이 로드된 이후에 처리했습니다.

③ 서버에 반영된 결제 상태를 확인해 결제 결과 페이지를 안내했습니다.

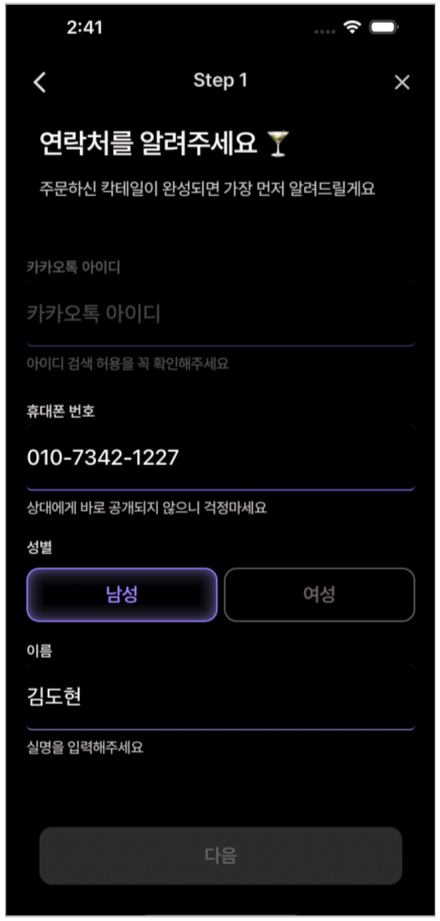
Other 시대팅 신청 과정 개선

기존 시즌의 피드백 개선을 핵심 목표로 삼았습니다.

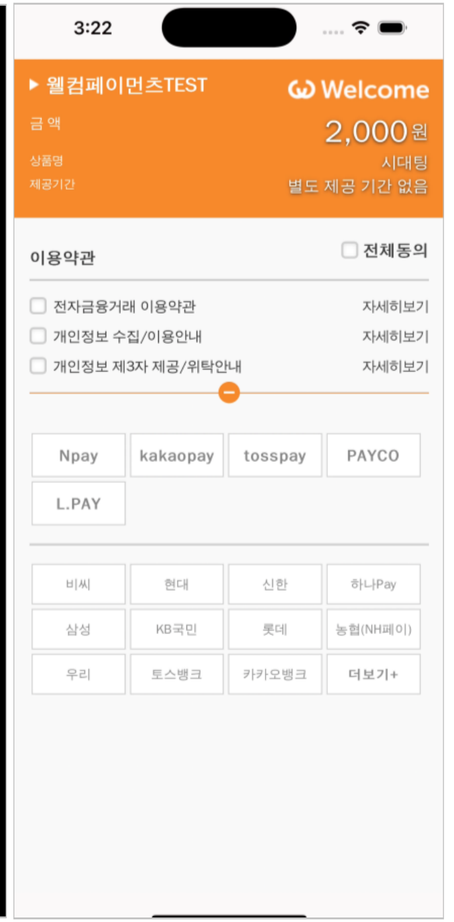
① 신청 중 이탈하면 처음부터 다시 입력해야 한다는 피드백이 있었습니다.

② useFunnel을 활용해 입력 단계를 나누어 단계별로 타입을 정의했습니다.

③ 이후 기존 입력 단계 상태에 따라 미완료 단계로 진입을 유도했습니다.



▲ 시대팅 신청 화면



▲ PortOne 결제 모듈 화면

프로젝트 성과

유료 결제 약 800건 · 결제 CS 10건 이하

결제 연동 모듈은 시즌8까지 재사용되고 있습니다.

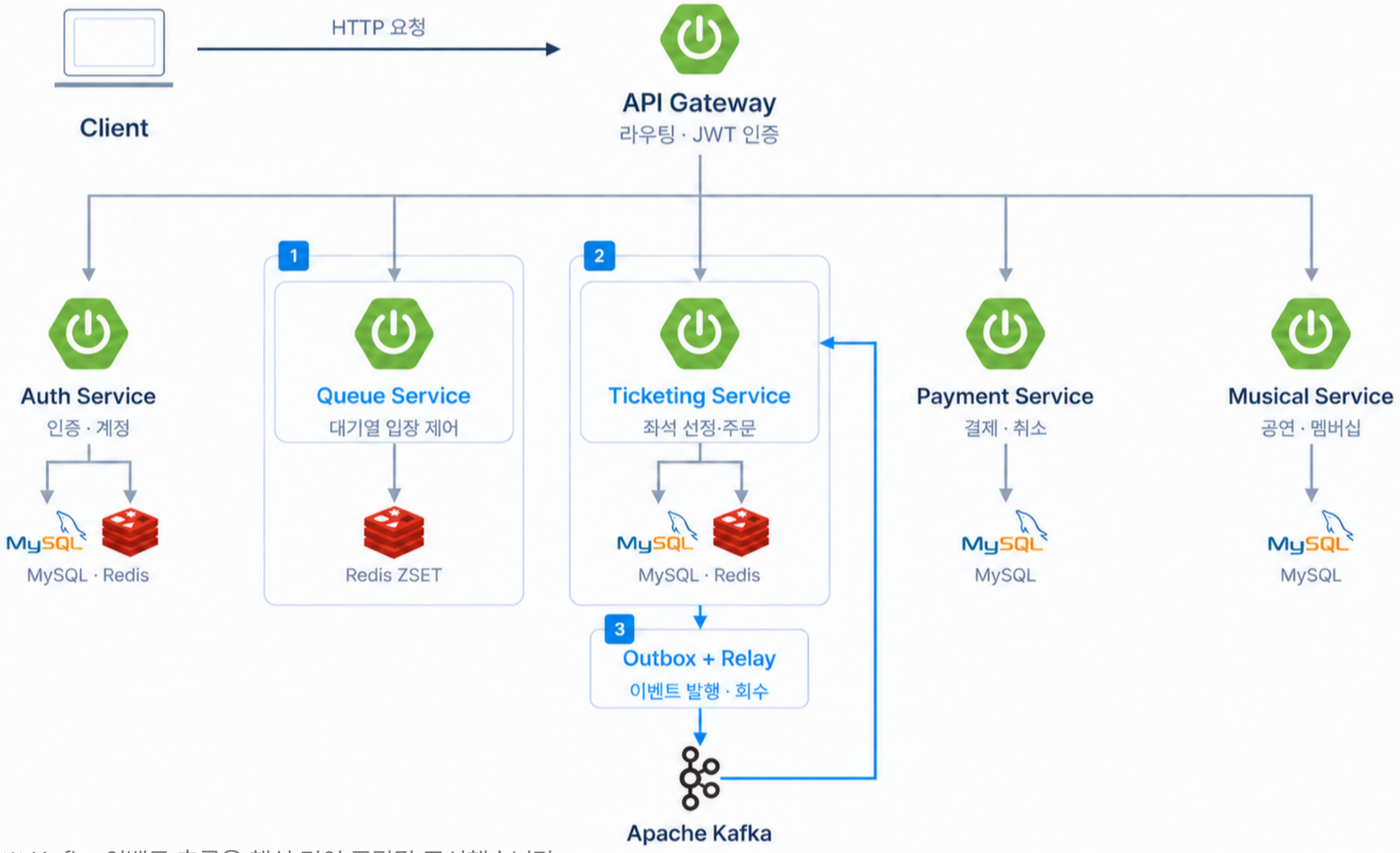
2. Truve

kt cloud TECH UP (26.01 – 26.04, 26.08 – 진행중) GITHUB 링크

뮤지컬 팬덤 플랫폼 | Java · Spring Boot · Redis · MySQL · Kafka

공연 예매, 멤버십, 아티스트 게시판, 채팅을 한 곳에 모은 뮤지컬 팬덤을 위한 예매 플랫폼입니다.

요청이 몰리는 순간에도 사용자가 선택한 좌석과 결제 결과를 신뢰할 수 있어야 한다고 생각했습니다.
이를 위해 요청량을 제어하고, 처리 실패와 이벤트 중복·유실에 대응하는 예매 흐름을 설계했습니다.



※ Kafka 이벤트 흐름은 핵심 기여 구간만 표시했습니다.

좌석과 결제 이후를 책임지며 설계를 개선했습니다.

1 대기열 기반 진입 제어

- Redis ZSET 기반 대기열 구성
- 순번별 폴링 주기 제어

2 좌석 선점과 실패 처리

- 주문·이벤트 기록 단일 트랜잭션으로 저장
- 실패 시 소유권 확인 이후 선점 해제

DB 저장에 성공해도 이벤트 전송은
실패할 수 있었습니다.

↓

3 이벤트 발행과 장애 복구

- Outbox 이벤트 기록과 Kafka 전송 분리
- 중단 작업 회수 이후 재발행 시도

실패 복구 흐름을 갖춘 뒤,
서버 시작 직후의 응답 지연을 개선했습니다.

↓

4 JVM 웜업으로 초기 응답 속도 개선

- 초기 응답 시간 약 83% 감소

Truve 설계 개선 ① – Outbox 패턴으로 상태 변경과 이벤트 기록의 일관성 확보

주문·티켓 상태 변경과 후속 이벤트를 같은 DB 트랜잭션에 저장하고, 발행에 실패한 이벤트는 다시 처리하도록 설계했습니다.

WHY

결제 완료가 주문과 티켓에 반영돼도, 좌석 판매 이벤트 전달에 실패하면 좌석 서비스에는 결제 결과가 반영되지 않을 수 있었습니다.
주문·티켓 상태 변경과 이벤트 기록을 같은 트랜잭션으로 저장하고, 저장된 이벤트를 별도로 발송했습니다.

HOW

주문·티켓 상태 변경 + Outbox 이벤트 저장 → 함께 커밋 → Kafka 발행 → 발행 결과 기록

실제 주문과 티켓의 데이터와 이벤트 기록을 함께 저장한 뒤, Kafka 발행은 DB 트랜잭션 밖에서 처리

① 상태 변경과 이벤트 기록의 동시 저장

주문 확정·티켓 발급 상태와 판매 이벤트를 함께 저장

저장 실패 시 전체 롤백

② 커밋된 이벤트를 기반으로 후속 처리 연결

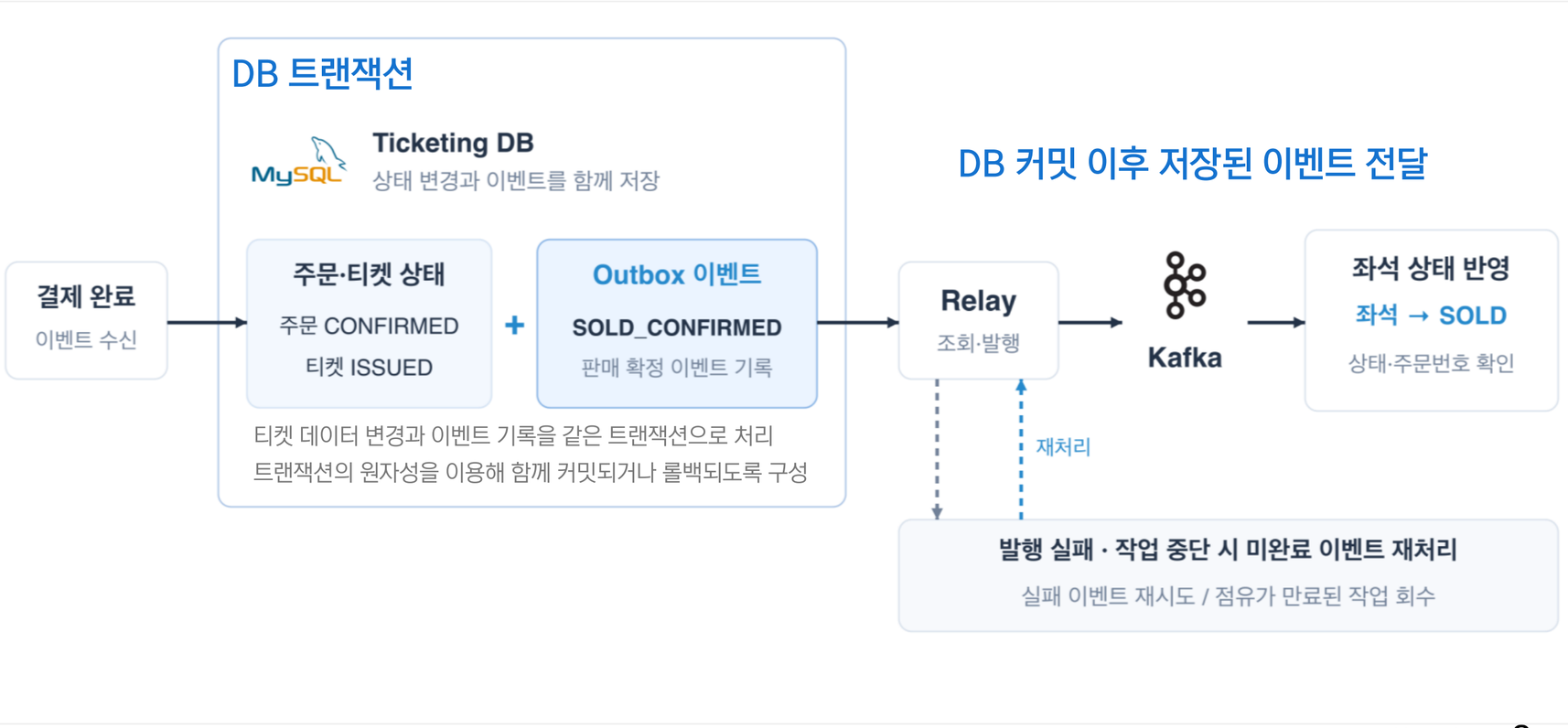
Relay가 커밋된 Outbox 이벤트를 읽어 Kafka로 발행

소비 측에서 이벤트를 처리해 좌석 판매 상태 반영

③ 발행 실패에 대비한 재처리와 멱등 처리

실패 이벤트 재시도·점유 만료 작업 회수

소비 측 상태·주문번호 확인으로 중복·지연 이벤트 처리



Truve 설계 개선 ② – JVM 웜업으로 초기 요청 지연 완화

서버 기동 직후의 실행 경로를 미리 준비하고, API 반복 호출로 JIT 컴파일을 유도해 초기 응답 지연을 줄였습니다.

WHY

예매 시작에 맞춰 요청이 몰리는 서비스에서 새 Pod의 초기 응답이 늦어지는 문제가 있었습니다.
기동이 완료돼도 좌석 조회와 응답 직렬화 등 실제 요청 경로의 실행 준비가 충분하지 않아, 트래픽을 받기 전에 해당 경로를 미리 실행할 필요가 있었습니다.

HOW

① HTTP 경로까지 웜업 확장

로컬에서 실제 API를 호출해

필터·MVC·Heartbeat 경로 사전 실행

② 반복 실행으로 JIT 최적화 유도

C2 최적화를 고려해 반복 횟수를 늘려 비교

C1·C2 컴파일 건수와 첫 응답 지연을 함께 확인

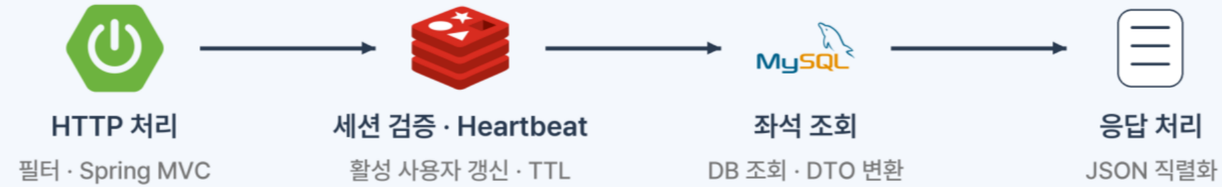
③ 준비 비용을 고려해 500회 선택

2,000회는 C2가 늘어도 첫 응답 지연 개선 효과 미미

준비 시간은 16.05초로 크게 증가

Warm-up 도입

DB·Redis 준비 + 실제 API 반복 호출



JVM 웜업 효과

localhost 환경에서 실제 API 호출

57.04 ms → **9.48 ms**

서버 기동 직후 응답 지연 83.4% 감소

컴파일·응답·준비 비용 비교

HTTP 호출	0회	100회	500회	2,000회
C1 (건)	10,511	12,184	13,810	14,829
C2 (건)	1,335	1,532	1,850	2,379
첫 응답 (ms)	57.04	11.06	9.48	9.56
준비 (초)	—	1.32	5.40	16.05

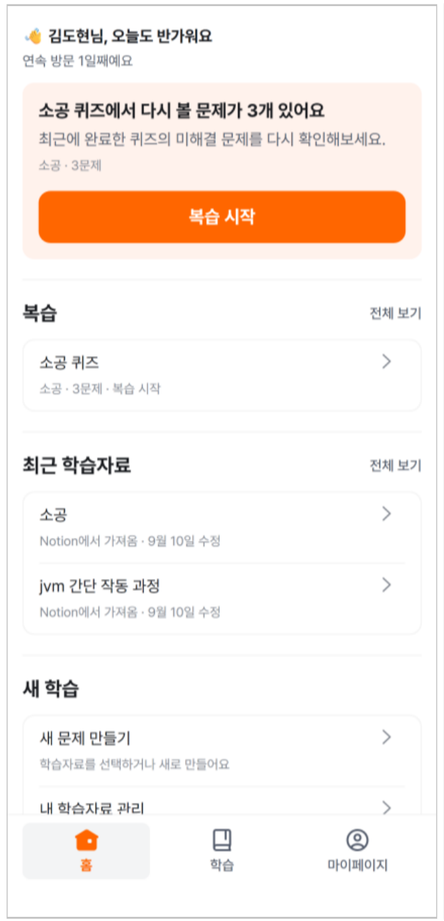
C1·C2: 측정 요청 시작 전 JVM 전체 컴파일 결과 생성 건수(nmethod)

3. NalQ

AI 활용 개인 프로젝트 (26.08 – 진행중) nalq.app 링크 GITHUB 링크

내 학습자료를 퀴즈와 복습으로 연결하는 플랫폼 | Java · Spring Boot · MySQL · Redis · React · AWS

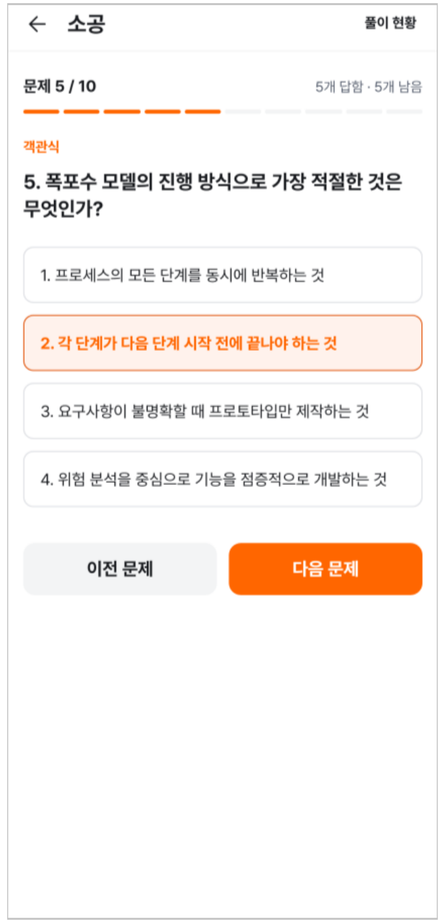
SSAFY 통학 시간을 학습에 활용하고자 'NalQ'를 개발했습니다. 기획부터 배포까지 혼자 진행하기 위해
AI에 맡길 구현 범위와 검증 기준을 문서화하고, 테스트와 코드 검토를 거쳐 2주 만에 MVP를 배포했습니다.



▲메인 화면



▲ 문제 생성 화면



▲ 문제 풀이 화면



▲ 채점 결과 화면

AI를 아래와 같이 활용했습니다.

1 AI가 참고할 맥락과 판단 기준 설정

- 기능 목적·사용자 흐름·예외 선정
- 설계 후보군의 trade-off 논의 후 확정
- PRD·UX·Contract·TRD 문서 작성

2 에이전트 역할과 구현 범위 지정

- 작업 별 에이전트 역할과 지침 설정
- 검토 가능한 작업 단위로 수정 범위 제한
- AGENTS.md를 통한 지침 제어

3 구현 결과를 확인할 검증 기준

- 로컬과 CI 동일 검증 기준 사용
- 생성 규칙과 실패 조건을 테스트로 확인
- 테스트 실패 시 반복 수정 루프 활용

4 디자인 일관성 확보

- 당근 SEED 디자인 시스템 오픈소스 활용
- DESIGN.md와 결합해 UI 일관성 유지

NaIQ 핵심 설계 – LLM 문제 생성 비동기 처리와 상태 관리

LLM 문제 생성을 전용 워커로 분리하고, 동시 작업 수와 결과 저장 조건을 제어했습니다.

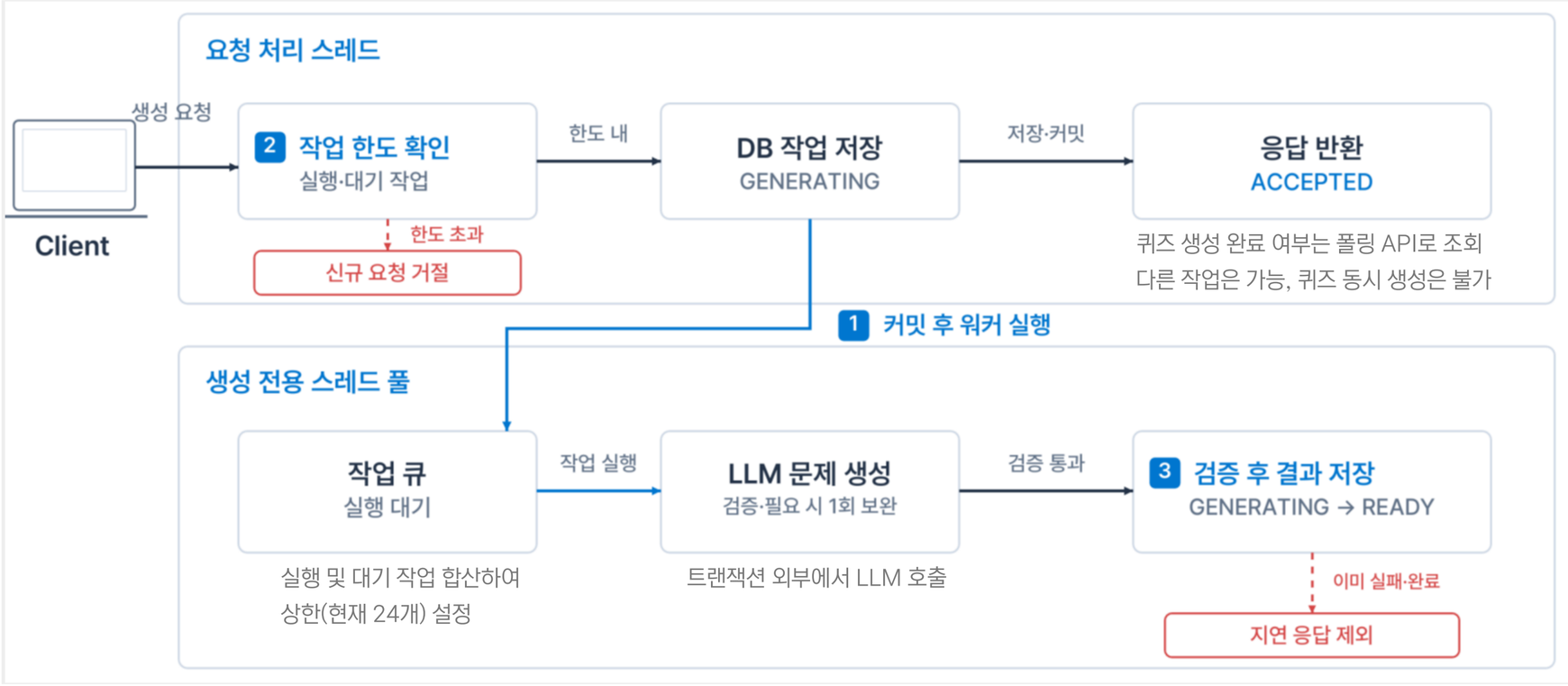
WHY

사용자가 문제 생성이 끝날 때까지 한 화면에서 기다리지 않고, 다른 학습을 이어갈 수 있도록 하고 싶었습니다.
생성이 끝날 때까지 대기 화면에 머물게 하면, 사용자는 다른 기능을 사용하지 못하고 기다려야 합니다.

HOW

작업 한도 확인 → DB 작업 저장 → 워커 실행→ 문항 및 작업 상태 검증 → 결과 저장

요청 처리와 문제 생성을 분리하여 사용자가 문제 생성을 요청하고 다른 기능을 사용할 수 있습니다.



① 요청 처리와 생성 작업 분리

DB 커밋 후 전용 워커에서 생성 실행

생성 완료 대기 없이 요청에 응답

② 동시 문제 생성 작업 수 제한

실행·대기 작업의 수를 합산해 상한 설정

한도 초과 시 신규 요청을 거절하고 재시도 안내 표시

③ 작업 상태 검증 후 저장

문항 검증 후 DB 행 잠금·상태 확인

GENERATING인 작업에만 결과 반영

사용자의 업무를 이해하고,

필요한 기능의 개발부터 안정적인 운영까지 책임지는

현대오토에버의 개발자가 되겠습니다.

지금까지 **지원자 김도현**이었습니다.

감사합니다.

이름: 김도현 이메일: kimdohyun032@gmail.com 전화번호: 010-7342-1227 GITHUB Link

목차로 이동